The University of the West Indies, Mona Campus

The **JET** System

Version 6.0

Lecturer: Mr Mathew Stone.

Studio Facilitator: Mr Mathew Stone.

Course: Introduction to Database Management

Course Code: COMP3161

Group: Spark

System: Spark—Course Management System

Due Date: May 1st, 2026.

Type: Education

Groups Members:

Melissa Grant	melissa.grant02@mymona.uwi.edu	**620156835**
Josiah-John Green	josiahjohngreen@mymona.uwi.edu	**620149044**
Justine Lewis	justine.lewis@mymona.uwi.edu	**620165751**
Shaqkeem Morrison	shaqkeem.morrison@mymona.uwi.edu	

Table of Contents

Table of Contents **1**

1.0. System Overview **2**

2.0. Entity Relationship Diagram **4**

3.0. Normalised Schema (3NF) **5**

3.1. Indexes 6

3.2. Reporting Views 6

4.0. Data Generation **8**

5.0. API Endpoints **10**

6.0. Setup & Deployment **13**

6.1. Local Setup 13

6.2. Railway & Vercel Deployment 14

1.0. System Overview

Spark is a course management platform — a RESTful backend designed to support an OURVLE-style frontend for students, lecturers, and administrators.

The system models the full academic lifecycle: accounts are created and authenticated, administrators create courses and assign a single lecturer to each, students enrol (subject to a 3–6 course constraint), and then lecturers run their courses day-to-day — posting calendar events, organising sectioned course content, maintaining discussion forums with threaded replies, creating assignments, and grading submissions. Each grade flows into a student's overall average, which is surfaced alongside five reporting views that answer questions about enrollment distribution, lecturer load, and student performance.

The project is built using raw SQL, normalised to 3NF, and seeded with at least 100,000 students spread across 200+ courses while respecting every enrollment and lecturer-load constraint.

The API is designed for a future React frontend deployed on Vercel and a Flask backend plus managed Postgres on Railway.

1.1. Architecture

The system is a three-tier application. The data tier is a PostgreSQL 16 database holding all entities and the five reporting views. The application tier is a Flask REST API organised into seven feature blueprints (auth, courses, events, forums, content, assignments, reports), with JWT-based authentication and role-based authorisation enforced at the route level. The presentation tier — not covered in this document — is a React/Vite frontend planned for Phase 2 that consumes this API.

1.1.1. Request Lifecycle

A client sends an HTTP request to a Flask route. For protected endpoints, the `@auth_required` decorator reads the `Authorisation: Bearer <jwt> header` verifies the JWT's signature with the server's secret, and attaches the decoded payload (user_id + role) to Flask's request-scoped `g.current_user`.

The `@role_required('admin')` decorator runs next and rejects the request if the user's role is not in the allowed set.

The route handler then executes a raw parameterised SQL query via psycopg2 helpers in `app/db.py`, which opens a connection, runs the query inside a RealDictCursor, commits or rolls back, and returns the JSON response.

2.0. Entity Relationship Diagram

The **ERD** below captures the full entity space of the system. Twelve entities, with all relationships labelled.

The diagram was designed in Assignment 1 and refined for the final project to add reply nesting on Post (self-reference via `parent_post_id`) and to make Grade one-to-one with Submission.

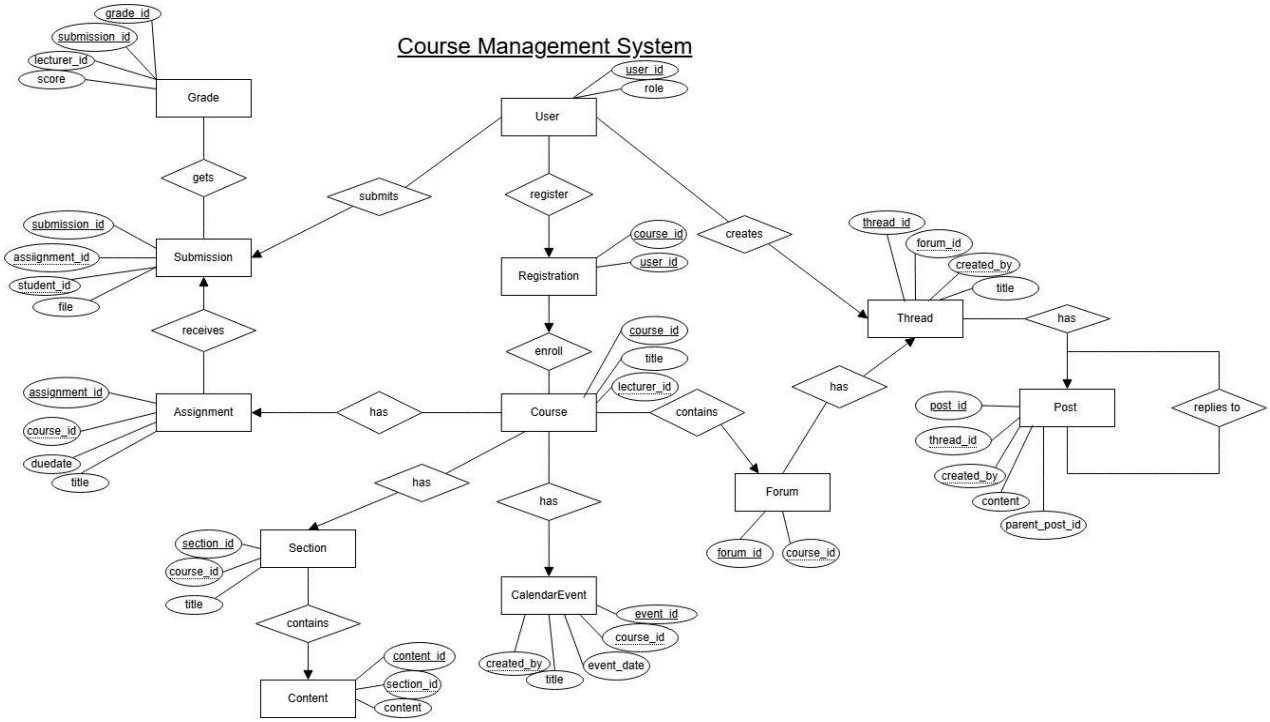

Figure 1. Spark ERD (Assignment 1 carry-over, refined for Phase 2).

3.0. Normalised Schema (3NF)

The **ERD** translates into 12 relational tables. Every table has a single-column primary key (or a composite natural key for the enrollment junction), and every non-trivial functional dependency lives at the key — no transitive or partial dependencies remain. All foreign keys cascade appropriately (CASCADE on tightly-owned child rows, SET NULL where a parent's removal should not erase history).

Table	**Primary Key**	**Notes**
users	user_id	Role-tagged: admin/lecturer/student
courses	course_id	FK to users(lecturer_id); one lecturer per course
registrations	(user_id, course_id)	Junction: enforces no double enrollment
calendar_events	event_id (serial)	FK to courses and users(created_by)
forums	forum_id (serial)	One-to-many with courses
threads	thread_id (serial)	Belongs to a forum
posts	post_id (serial)	Self-FK via parent_post_id for nested replies
sections	section_id (serial)	Groups course content; display_order for sorting

content_items	content_id (serial)	ENUM(link|file|slide)
assignments	assignment_id (serial)	CHECK (max_score > 0)
submissions	submission_id (serial)	UNIQUE(assignment_id, student_id)
grades	grade_id (serial)	UNIQUE(submission_id); score 0-100

3.1. **Indexes**

Sixteen B-tree indexes cover every foreign-key column and every column referenced by a non-trivial WHERE clause in the API. Notable ones include:

1. `idx_calendar_date` (supports the student-events-on-date endpoint)
2. `idx_posts_parent` (supports reply trees)
3. `idx_submissions_student` (supports the student-average computation).
4. Indexes are declared explicitly in `schema.sql.`

3.2. **Reporting Views**

The five reporting queries required by the spec are materialised as persistent views, so the API reports layer is a thin pass-through, and the query logic is inspectable in the schema file. The views are:

`1. v_courses_50_plus_students`

2. v_students_5_plus_courses

3. v_lecturers_3_plus_courses

4. v_top_10_enrolled_courses

5. and v_top_10_students_by_average.

4.0. Data Generation

The file `database/generate_seed.py` produces a single `seed.sql` containing multi-row INSERT batches for every table. The generator is deterministic (fixed random seed) and satisfies every data-volume constraint from the spec by construction.

Constraint	**How is it enforced?**
>100,000 students	`NUM_STUDENTS = 100_000 (configurable)`
>200 courses	`NUM_COURSES = 200 (configurable)`
Student enrols in 3—6 courses	`random.randint(3, 6) samples per student`
Course has10 members	`Post-pass enforcement loop adds members if short`
The lecturer teaches 1—5 courses	`Each lecturer is assigned 1 course; the rest are distributed with a cap of 5`
Every lecturer teaches >= 1 course	`Initialisation pass guarantees 1 per lecturer`

The generator also populates forums, threads, posts (with nested replies), sections, content items, calendar events, assignments, submissions, and grades at realistic fixture volumes, so every API endpoint has data to return when the project is demoed.

5.0. API Endpoints

Thirty-four feature endpoints plus two health endpoints, organised into seven Flask blueprints.

The full set is importable as a Postman collection:

`(postman/Spark_API.postman_collection.json).`

Protected endpoints are marked with the role(s) that may access them.

#	Method	Path	Access
1	POST	`/register`	public
2	POST	`/login`	public
3	POST	`/courses`	admin
4	GET	`/courses`	public
5	GET	`/students/{id}/courses`	public
6	GET	`/lecturers/{id}/courses`	public
7	POST	`/courses/{id}/assign-lecturer`	admin
8	POST	`/courses/{id}/register`	student
9	GET	`/courses/{id}/members`	public

10	POST	`/courses/{id}/events`	lecturer/admin
11	GET	`/courses/{id}/events`	public
12	GET	`/students/{id}/events?date=YYYY-MM-DD`	public
13	GET	`/courses/{id}/forums`	public
14	POST	`/courses/{id}/forums`	authenticated
15	GET	`/forums/{id}/threads`	public
16	POST	`/forums/{id}/threads`	authenticated
17	GET	`/threads/{id}/posts`	public
18	POST	`/threads/{id}/posts`	authenticated
19	GET	`/courses/{id}/sections`	public
20	POST	`/courses/{id}/sections`	lecturer/admin
21	POST	`/sections/{id}/content`	lecturer/admin
22	GET	`/courses/{id}/content`	public
23	POST	`/courses/{id}/assignments`	lecturer/admin
24	GET	`/courses/{id}/assignments`	public

25	POST	`/assignments/{id}/submit`	student
26	GET	`/assignments/{id}/submissions`	lecturer/admin
27	POST	`/submissions/{id}/grade`	lecturer/admin
28	GET	`/students/{id}/average`	public
29	GET	`/reports/courses/50-plus-students`	public
30	GET	`/reports/students/5-plus-courses`	public
31	GET	`/reports/lecturers/3-plus-courses`	public
32	GET	`/reports/courses/top-10-enrolled`	public
33	GET	`/reports/students/top-10-averages`	public

6.0. Setup & Deployment

6.1. **Local Setup**

1. Create the database and load the schema:

```
createdb spark
psql spark -f database/schema.sql
```

2. Generate and load the seed data:

```
cd database && python3 generate_seed.py
psql spark -f seed.sql
```

3. Start the backend:

```
cd backend
pip install -r requirements.txt
cp .env.example .env # edit with your DB creds
python3 run.py
```

6.2. **Railway & Vercel Deployment**

The backend deploys to Railway via the included **Procfile**.

Add a PostgreSQL plugin on Railway, copy its credentials into the service's environment variables `(DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD)`, set **`JWT_SECRET`** to a long random string, and set the service's Root Directory to **`backend/`**.

Railway auto-runs `gunicorn wsgi: app`. Load the schema and seed from a local machine, pointing psql at the Railway connection URL.

The React frontend will deploy to Vercel and point its **`VITE_API_URL`** at the Railway public URL.